IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2025, No. 4, pp. 795–816. DOI:10.46586/tches.v2025.i4.795-816

A New Trick for Polynomial Multiplication

A verified CRT polymul utilizing a monomial factor

Chun-Ming Chiu[1], Bo-Yin Yang[2] and Bow-Yaw Wang[2]

[1] National Taiwan University, Taipei, Taiwan, r12922085@csie.ntu.edu.tw
[2] Academia Sinica, Taipei, Taiwan, {byyang,bywang}@iis.sinica.edu.tw

Abstract. In this paper we present a novel transformation strategy for polynomial multiplications and apply it to NTRU Prime, specifically the parameter sets `sntrup761` and `ntrulpr761` working in the ring $\mathbb{Z}_{4591}[x]/\langle x^{761}-x-1\rangle$. To evaluate the practicality of our idea, we implemented the algorithm in C++ with ARM Neon intrinsics. By further exploiting the various optimization opportunities in the transformation process, we achieve state-of-the-art performance on Cortex-A72.
Because of the aggressively lazy modular reduction strategy, overflows are of serious concern. Such errors in an optimized implementation are notoriously difficult to detect using traditional test vectors. To this end, the compiled binary file is formally verified using the tool CryptoLine. We use all the features in the current version of CryptoLine. This includes the Integer Set Library for range checking, plus the Logical Equivalence Checking to verify the correctness of the binary produced with the most optimized compiler setting by showing it as being equivalent to a binary from a less optimized compilation.

Keywords: CRT polymul · NTT · monomial modulus · NTRU Prime

1 Introduction

Optimized implementations of the Number-Theoretic Transform (NTT) had received a lot of attention since 2016, especially during most of the NIST postquantum standardization process (competition) because it is a main component in implementing lattice-based postquantum cryptosystems. Furthermore, in NTRU-type lattice-based cryptosystems (where the public key is a quotient), the polymul (polynomial multiplication) operation consumes most of the cycles.

After Crystals-Kyber was crowned as the sole KEM winner in the NIST "non-competition", the crypto community's interest in implementing NTT variants seems to have waned. Part of that is because in ML-KEM and ML-DSA (formerly Kyber and Dilithium), an NTT is baked into the specifications, enforcing most of the NTT on the implementer. Part of that is because in the NewHope-Kyber line of cryptosystems (where the public key is a product-plus-error) most of the cycles on a big modern CPU are consumed by hash function calls, which cuts into the potential gain of further NTT optimizations. Part of that is because due to the effort poured into NTT implementations, the current ones are actually quite good, which also limits any gains from further investigation.

Despite this, we believe that polynomial arithmetic techniques are still valuable. In part, polynomial multiplication *is* a foundational implementation technique; in part, crypto agility is always a good idea — we should not stop implementing any lattice-based cryptosystem other than the NIST standards, because the popular 25519 curve shouldn't exist by the same reasoning. At least OpenSSH agrees because they are likely to continue to support Streamlined NTRU Prime (`sntrup761`) in the near future.[1]

[1] https://mailarchive.ietf.org/arch/msg/ssh/eDaZrtmNnLwbQo_KmIi7PR_I6tU/

Licensed under Creative Commons License CC-BY 4.0.
Received: 2025-04-15 Accepted: 2025-06-15 Published: 2025-09-05

Contributions: Fast Fourier Transforms and NTTs most often use moduli of the form $x^n - 1$, followed by the cyclotomic $\Phi_n(x^k)$ — ML-KEM/DSA (and Saber) use $\Phi_{512}(x)$, FN-DSA (Falcon) uses $\Phi_{\{1024,2048\}}(x)$. For more unusual examples, Lyubashevsky–Seiler [LS19] uses $\Phi_{2304}(x)$ for NTTRU, Bernstein et al. in [BBCT22] and Hwang [Hwa24] respectively uses $(x^{1024}+1)(x^{512}-1)$ and $\Phi_{17}(x^{96})$ for `sntrup761` (cf. Section 3.1)

We introduce NTT-based multiplications using polynomial moduli that contain a factor of x^m and show that it can be faster than the state of the art for `sntrup761` and demonstrate this on CPUs including the Cortex-A72, used in the popular Raspberry Pi 4. We also introduce lazy reductions that substitute some shifts for multiplications. The program in C++ with intrinsics is shown to be correct (making it more deployable) via formal verification using the CryptoLine toolkit. On what is by now a highly optimized piece of code, we believe the speed-up we get is very remarkable. This work can also be treated as an instructional example and reference in engineering high-performance and high-assurance polynomial multiplication using unorthodox NTT techniques. In the process, we also showcase CryptoLine as a tool for the programmer. There are few if any "in-the-wild" complex machine-optimized assembly code scheduled by a non-human that were verified and this stretches the horizon for such tools.

Related Work: NTRU Prime Since the NTRU prime field is naturally not NTT-friendly, many tricks had been used to do fast multiplication. For methods retaining the integer modulus, Initially Toom-Cook was used. The change-of-modulus-into-NTT method which was mentioned as early as (among other places) [Ber01] was first introduced to NTRU Prime in the submission [CHK+21] (along with the very rare Rader-17, in the ring $\mathbb{Z}_{4591}[x]/\langle x^{1530}-1\rangle$) and lattice-based crypto at large in [ACC+21]. While the code package with [BBC+20] already contained such code, [BBCT22] seems to be the first paper to use $\mathbb{Z}_{4591}[x]/\langle(x^{1024}+1)(x^{512}-1)\rangle$ in an truncated Schönhage. [HLY24] continued to use Rader-17 with $\mathbb{Z}_{4591}[x]/\langle x^{1632}-1\rangle$ (coming down to length-16 convolutions), and [Hwa24] improved this to T(runcated)Rader-17 and the smaller ring $\mathbb{Z}_{4591}[x]/\langle\Phi_{17}(x^{96})\rangle$. For NTT multiplications switching the integral modulus, [ACC+21] had an incomplete NTT with $\mathbb{Z}_{6984193}[x]/\langle x^{1536}-1\rangle$. *Despite all these papers, the use of a monomial modulus for CRT multiplication including a multidimensional Good's trick seems to be quite new.*

Related Work: Other There are many papers over the last few years about polynomial multiplications in Kyber, Dilithium, Saber, and NTRU, the NIST lattice finalists. The most recent advances seem to be [CHK+21] introduced the use of Cooley-Tukey butterflies going both for Forward and Inverse NTTs; [Sei18], [BHK+22] and [HZZ+22] introduced signed Montgomery, Barrett and Plantard multiplications respectively. For non-NTT polymul, [CCHY23] describes the state of the art (Toom-based TMVP methods). There are a few papers dealing with verified lattice-based crypto code, such as [ABB+23, AOB+24, HLS+22], all of them using mostly 2-way NTTs. There are also many articles in the vein of [BBF+21], but those are protocol-level verification. Verification of a "non-standard" CRT-NTT multiplication is new as far as we are able to determine, and so is the use of rightshifts replacing divisions including their associated range verifications.

2 Preliminaries

2.1 NTRU Prime

The NTRU Prime family comprises an NTRU variant PKE (public-key encryption) and a Ring-Learning-with-Rounding variant PKE, using ternary errors and a finite field for a polynomial ring, which are made into two KEMs (Key Establishment Methods) via a Fujisaki-Okamoto Transform [FO99]. There are a variety of possible parametrizations, but

the main recommended variants uses the finite field $\mathbb{Z}_{4591}[x]/\left\langle x^{761} - x - 1)\right\rangle \cong \mathbb{F}_{4591^{761}}$ as the polynomial ring. We refer the reader to [BBC+20] for details. It suffices to mention that *the main operation during encapsulation and decapsulation is multiplying a random-looking polynomial with a ternary polynomial.*

2.2 Modular Reductions and Multiplications

In the context of lattice-based crypto, modular reductions means to select a smaller-in-magnitude congruential representative. Often the reductions are "lazy" in that the canonical representative mod q (whether between $\pm q/2$ or in $[0, q-1]$) is not necessarily obtained. The objective is simply to reach an integer small enough in magnitude that arithmetic can continue to happen without overflowing. A modular multiplication (mulmod) is to find a small enough congruential representative of a product, which again need not be canonical. Currently there are three standard ways to perform modular multiplications: Barrett [BHK+22], Montgomery [Sei18] and Plantard [HZZ+22].

2.2.1 (Subtractive) Montgomery Reduction and Multiplication

[Sei18] notes that given X and a suitable two-power R, we precompute $q' = q^{-1} \bmod R$, and now can obtain $XR^{-1} \bmod q$ by first computing $\ell = Xq' \bmod R$, then (since $R|(X - \ell q)$) $XR^{-1} \equiv (X - \ell q)R^{-1} \equiv X_h - [\ell q]_h \pmod q$, where the "high half" $[\cdot]_h = \lfloor \cdot / R \rfloor$. This is advantageous whenever there a "high-half" product. For known b with $|b| < \frac{q}{2}$, we can simplify a mulmod further to $ab \equiv [a \cdot B]_h - [q \cdot [a \cdot B']_l]_h \pmod q$ with $2 \times \text{high} + 1 \times \text{low}$ mults using precomputed $B = bR \bmod^{\pm} q$, $B' = Bq' \bmod^{\pm} R$. (Here $[xy]_l = xy \bmod^{\pm} R$.)

2.2.2 Signed Barrett Reduction and Multiplication

We say that the function $[\![\cdot]\!] : \mathbb{R} \to \mathbb{Z}$ is an *integer approximation* if $|x - [\![x]\!]| \le 1$, and $a \bmod^{[\![\cdot]\!]} b$ is then defined to be $a - [\![a/b]\!]b$. When $[\![\cdot]\!]_0$, $[\![\cdot]\!]_1$ are integer approximations, a representative of $ab \bmod q$ is available via $ab - eq \equiv ab \pmod q$, where $e = \left[\!\!\left[\frac{a[\![\frac{bR}{q}]\!]_0}{R} \right]\!\!\right]_1 \approx \frac{ab}{q}$. Becker et al. [BHK+22] showed[2] that

$$|ab - eq| \le \frac{1}{R}\left[|a| \left|\bmod^{[\![\cdot]\!]_0} q\right| + \left|\bmod^{[\![\cdot]\!]_1} R\right| q \right] , \tag{1}$$

where $\left|\bmod^{[\![\cdot]\!]} N\right|$ denotes $\max_a \left|a \bmod^{[\![\cdot]\!]} N\right|$ for the integer approximation $[\![\cdot]\!]$. They also noted that for fixed b, one can and should precompute $\bar{b} = [\![bR/q]\!]_0$. For implementation in Neon-like instruction sets, the 16-bit `sqrdmulh` computes $\lfloor 2a\bar{b}/2^{16} \rceil = \lfloor a\bar{b}/2^{15} \rceil$, so the most natural choice is $R = 2^{15}$ and $[\![\cdot]\!]_0 = [\![\cdot]\!]_1 = \lfloor \cdot \rceil$, with precomputed $\bar{b} = \lfloor 2^{15} b/q \rceil$.[3] Under this choice, Equation 1 guarantees $|ab - eq| \le q$ for any 16-bit signed integer a (since $|a| \le 2^{15}$). This allows us to compute $ab - eq$ using only 16-bit arithmetics, specifically one `mul` for the lower-half product $[ab]_l$, one `sqrdmulh` for e, and lastly one fused-multiply-subtract `mls` for $[[ab]_l - eq]_l = [ab - eq]_l = ab - eq$. Note that we are effectively computing in $\mathbb{Z}_{2^{16}}$, so *overflows are destined but completely irrelevant.*

When $b = 1$ we have a standard single-limb Barrett reduction. However, when we desire the canonical representative, we often specialize to a constant $q' = \lfloor 2^{\ell-1} R/q \rceil$ where $2^{\ell+1} > q > 2^{\ell}$, such that $\frac{R}{2} > q' > \frac{R}{4}$. This let us approximate $\lfloor a/q \rceil$ as $\lfloor 2^{1-\ell}[aq']_h \rceil$, computable as a Neon-available rounded right-shift $\left(\left([aq']_h + 2^{\ell-2}\right) \gg (\ell - 1)\right)$.

[2] They showed only the case where $[\![\cdot]\!]_1 = \lfloor \cdot \rceil$, and stated their bound [BHK+22, Cor. 2] without the notion of $\left|\bmod^{[\![\cdot]\!]} N\right|$. This notion and the generalized bound are first introduced in [HKS23].

[3] In [BHK+22] they chose $R = 2^{16}$, $[\![\cdot]\!]_0 = 2\lfloor \cdot/2 \rceil$ ("round to even") and $[\![\cdot]\!]_1 = \lfloor \cdot \rceil$, with precomputed $\hat{b} = [\![2^{16} b/q]\!]_0/2$. Despite the different appearance, both choices lead to exactly the same computation.

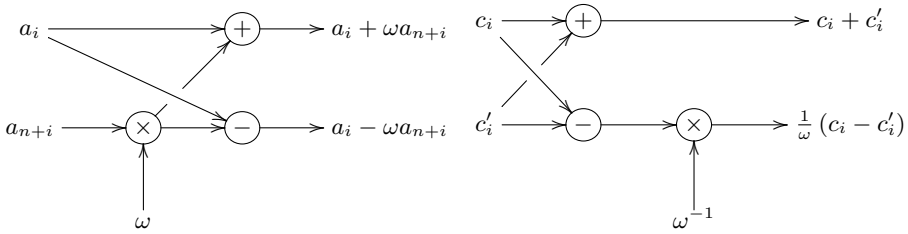
2.3 CRT and FFTs/NTTs

The CRT (Chinese Remainder Theorem) is a standard theorem of ring theory that relates quotient rings for ideals $I_1, I_2 \subset R$ as follows: $R/(I_1 \times I_2) \cong R/I_1 \times R/I_2$. This holds for PIDs (Principal Ideal Domains) which include the integers and polynomials over a field.

The standard Cooley-Tukey FFT (Fast Fourier Transform) or NTT (Number Theoretic Transform) is exactly the CRT instance that states the following to be an isomorphism

$$\phi : \frac{\mathbb{Z}_q[x]}{\langle x^{2n} - \omega^2 \rangle} \to \frac{\mathbb{Z}_q[x]}{\langle x^n - \omega \rangle} \times \frac{\mathbb{Z}_q[x]}{\langle x^n + \omega \rangle}, \phi : f \mapsto (f \bmod (x^n - \omega), f \bmod (x^n + \omega))\,;$$

or (as a map between coefficient vectors) $f : (a_0, a_1, \ldots, a_{n-1}, a_n, a_{n+1}, \ldots, a_{2n-1}) \mapsto (a_0 + \omega a_n, a_1 + \omega_{n+1}, \ldots, a_{n-1} + \omega a_{2n-1}) \times (a_0 - \omega a_n, a_1 - \omega_{n+1}, \ldots, a_{n-1} - \omega a_{2n-1})$. Mapping $(a_0, a_n) \mapsto (a_0 + \omega a_n, a_0 - \omega a_n)$ is precisely the Cooley-Tukey butterfly (Figure 1a). Its almost-inverse (modulo a division by 2) is the Gentleman-Sande butterfly (Figure 1b).



(a) Cooley–Tukey (CT) Butterfly **(b)** Gentleman–Sande (GS) Butterfly

Figure 1: Butterflies in NTT

If we repeat the above process enough times, we get the NTTs used for ML-KEM/DSA. In short CT and GS butterflies suffices for ML-KEM and ML-DSA, but for NTRU Prime, maximal speed requires some other decompositions, which we shall describe below.

2.3.1 Good–Thomas

Good and Thomas [Goo58, Tho63] proposed turning an NTT for a modulus of $x^{p_0 p_1} - 1$ with coprime p_0, p_1 into a two-dimensional NTT by substituting $x = yz$ with $y^{p_0} = z^{p_1} = 1$. More precisely, the univariate polynomial $\sum_{i=0}^{p_0 p_1} a_i x^i$ becomes a bivariate $\sum_{j=0}^{p_0-1} \sum_{k=0}^{p_1-1} a_{R(j,k)} y^j z^k$, where $R(j,k) = \left(j p_1 \left(p_1^{-1} \bmod p_0\right) + k p_0 \left(p_0^{-1} \bmod p_1\right)\right) \bmod p_0 p_1$ is the "Ruritanian permutation". With the "Good's Trick", we can do the p_0-NTT (in the y-axis) and the p_1-NTT (in the z-axis) independently. In particular, both these FFTs are in a ring modulo $y^{p_0} - 1$ and $z^{p_1} - 1$, which makes things a lot simpler and more repetitive (i.e., with fewer constants to load).

2.3.2 Rader

Rader's FFT [Rad68] offers a way to compute p-FFT for odd primes p by reducing the problem into a size-$(p-1)$ convolution. We can then transform the problem back to FFTs, where their size $(p-1)$ will at least have 2 as a factor. The basic p-ary transform procedures, usually also called *butterflies*, can be derived from the Rader procedure. For small p, it is worthwhile to optimize the resulting butterfly by hand using some equalities and we do so using various cyclotomic equalities such as $\sum_{i=1}^{p-1} \omega_p^i = -1$. In our algorithm, we use radix-3 and -5 butterflies as basic building blocks. We write $F_i = \sum_{j=0}^{p-1} f_j \omega_p^{ij}$, $\hat{F}_i = F_i - f_0$.

Radix-3 To avoid clutter, we abbreviate ω_3 to ω during our discussion of the radix-3 case. Here, Rader's trick gives the equation $(\hat{F}_1, \hat{F}_2) = (f_1, f_2) * (\omega, \omega^2)$. Using FFT to compute the convolution, we get

$$a_0 = \hat{F}_1 + \hat{F}_2 = (f_1 + f_2)(\omega + \omega^2) = -(f_1 + f_2) \tag{2a}$$
$$a_1 = \hat{F}_1 - \hat{F}_2 = (f_1 - f_2)(\omega - \omega^2) \tag{2b}$$
$$2\hat{F}_1 = a_0 + a_1 \tag{2c}$$
$$2\hat{F}_2 = a_0 - a_1 \,. \tag{2d}$$

There is no need to remove the factor of 2 here. We can absorb this factor into all of the outputs F_0, F_1, F_2. With this in mind, we have

$$2F_0 = 2(f_0 + f_1 + f_2) \tag{2e}$$
$$2F_1 = 2f_0 + 2\hat{F}_1 \tag{2f}$$
$$2F_2 = 2f_0 + 2\hat{F}_2 \,. \tag{2g}$$

Note that we have the intermediate value $f_1 + f_2$ during (2a), which can be reused during (2e). Also, rather than adding $2f_0$ into $2\hat{F}_1$ and $2\hat{F}_2$ at (2f) and (2g), we can save an addition by letting $a_0 \leftarrow a_0 + 2f_0$ before computing (2c) and (2d). The resulting algorithm can be drawn as a butterfly shown in Figure 2.

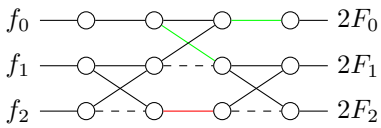


Figure 2: Our radix-3 butterfly, where the green and red edges represent multiplication by 2 and $\omega_3 - \omega_3^2$ respectively.

Radix-5 During our discussion of the radix-5 case, we abbreviate ω_5 to ω. This time, Rader's trick gives the equation $(\hat{F}_1, \hat{F}_3, \hat{F}_4, \hat{F}_2) = (f_1, f_2, f_4, f_3) * (\omega, \omega^3, \omega^4, \omega^2)$. Applying 2-NTT once gives

$$(a_0, a_1) = (\hat{F}_1 + \hat{F}_4, \hat{F}_3 + \hat{F}_2) = (f_1 + f_4, f_2 + f_3) * (\omega + \omega^4, \omega^3 + \omega^2)$$
$$(a_2, a_3) = (\hat{F}_1 - \hat{F}_4, \hat{F}_3 - \hat{F}_2) = (f_1 - f_4, f_2 - f_3) \mathbin{\overline{*}} (\omega - \omega^4, \omega^3 - \omega^2) \,,$$

where the operator $\overline{*}$ denotes negacyclic convolution or, equivalently, multiplication modulo $x^2 + 1$ when (u, v) is identified with polynomial $u + vx$. Note that while we are able to decompose the cyclic convolution again:

$$b_0 = a_0 + a_1 = (f_1 + f_4 + f_2 + f_3)(\omega + \omega^4 + \omega^3 + \omega^2) = -(f_1 + f_4 + f_2 + f_3)$$
$$b_1 = a_0 - a_1 = (f_1 + f_4 - f_2 - f_3)(\omega + \omega^4 - \omega^3 - \omega^2)$$
$$2a_0 = b_0 + b_1$$
$$2a_1 = b_0 - b_1 \,,$$

the absence of square root of -1 inside $\mathbb{Z}_q = \mathbb{Z}_{4591}$ forbids us to do the same to the negacyclic convolution. Nonetheless, by falling back to Karatsuba (and rescaling everything here by

2 to meet the factor in the cyclic part), we obtain

$$\begin{aligned}
2c_0 &= (f_1 - f_4) \cdot 2(\omega - \omega^4) \\
2c_\infty &= (f_2 - f_2) \cdot 2(\omega^3 - \omega^2) \\
2c_1 &= (f_1 - f_4 + f_2 - f_3) \cdot 2(\omega - \omega^4 + \omega^3 - \omega^2) \\
2a_2 &= 2c_0 - 2c_\infty \\
2a_3 &= 2c_1 - 2c_0 - 2c_\infty \,.
\end{aligned}$$

Combined together, these grant us

$$\begin{aligned}
4F_0 &= 4(f_0 + f_1 + f_2 + f_3 + f_4) \\
4F_1 &= 4f_0 + 4\hat{F}_1 = 4f_0 + 2a_0 + 2a_2 \\
4F_4 &= 4f_0 + 4\hat{F}_4 = 4f_0 + 2a_0 - 2a_2 \\
4F_3 &= 4f_0 + 4\hat{F}_3 = 4f_0 + 2a_1 + 2a_3 \\
4F_2 &= 4f_0 + 4\hat{F}_2 = 4f_0 + 2a_1 - 2a_3 \,.
\end{aligned}$$

Corresponding to the radix-3 case, by eliminating the common subexpression $f_1+f_2+f_3+f_4$, and replacing the four additions with $4f_0$ with one in $b_0 \leftarrow b_0 + 4f_0$, we end up with a radix-5 butterfly shown in Figure 3, using 4 modular multiplications in total. Though be aware that it is impossible to do every multiply-by-4 edge by simply left-shift by 2: One can easily see that overflows are destined to occur in some of the nodes, even if $f_0, \ldots, f_4$ are all in the tightest range $[-2295, 2295]$. One solution to this is treat this as a Barrett multiplication by 4, except with the actual multiply by 4 replaced with a shift left 2.

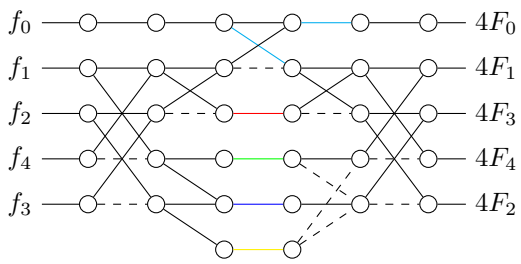


Figure 3: Our radix-5 butterfly, where the cyan, red, green, blue and yellow edges represent multiplication by 4, $\omega_5 + \omega_5^4 - \omega_5^3 - \omega_5^2$, $2(\omega_5 - \omega_5^4)$, $2(\omega_5 - \omega_5^4 + \omega_5^3 - \omega_5^2)$ and $2(\omega_5^3 - \omega_5^2)$ respectively.

2.4 Doing iNTT as NTT

The n-NTT is a case of the discrete Fourier Transform $\hat{a}_j = \sum_{i=0}^{n-1} \omega^{ij} a_i$, where ω is a principal root of unity of order n. The inverse of that is $a_i = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{-ij} \hat{a}_j$. This (aside from the $\frac{1}{n}$ and having to rearrange the coefficients) is exactly the NTT but with ω^{-1} instead of ω. The upshot is that one can always do iNTTs as if they are NTTs. The by now standard trick of using CT butterflies both ways for ML-KEM (to avoid intermediate reductions) can be explained thus instead of using the Twisted (Gentleman-Sande) NTT.

2.5 Weighted Convolutions and Toeplitz Matrix-Vector Products

At the end of our reductions we typically come down to a product $h = fg \bmod (x^{16} - w)$ for degree-15 (or lower) polynomials f, g. When $w = 1$ this is the standard convolution

and $w = -1$ it is a negacyclic convolution. When $w \neq \pm 1$, we can write this as

$$\begin{bmatrix} h_0 \\ h_1 \\ h_2 \\ \vdots \\ h_{15} \end{bmatrix} = \begin{bmatrix} g_0 & wg_{15} & wg_{14} & \cdots & wg_1 \\ g_1 & g_0 & wg_{15} & \cdots & wg_2 \\ g_2 & g_1 & g_0 & \cdots & wg_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ g_{15} & g_{14} & g_{13} & \cdots & g_0 \end{bmatrix} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{15} \end{bmatrix}.$$

In other words, it is a Toeplitz Matrix to Vector Product (TMVP). When w has a square root we can use a Cooley-Tukey NTT to reduce it to two smaller TMVPs. Otherwise, as in Karatsuba, we need three 8×8 TMVPs using the methods in (e.g.) [CCHY23]:

$$\begin{bmatrix} C_0 \\ C_1 \end{bmatrix} = \begin{bmatrix} A_0 & A_{-1} \\ A_1 & A_0 \end{bmatrix} \begin{bmatrix} B_0 \\ B_1 \end{bmatrix} \rightarrow \begin{cases} C_0 &= (A_{-1} - A_0)B_1 + A_0(B_0 + B_1) \\ C_1 &= A_0(B_0 + B_1) + (A_1 - A_0)B_0. \end{cases}$$

Note that submatrices of a Toeplitz matrix are also Toeplitz.

3 Implementation

In the following, we denote the two input polynomial as $A(x) = \sum_{i=0}^{760} a_i x^i$ and $B(x) = \sum_{i=0}^{760} b_i x^i$. Our algorithm is for big-by-big multiplications, that is, we can only assume that $a_i, b_i \in [-2295, 2295]$ for each i. Similarly, the output polynomial, before and after reducing modulo $x^{761} - x - 1$, are respectively denoted as $C^{\text{full}}(x) = \sum_{i=0}^{1520} c_i^{\text{full}} x^i$ and $C(x) = \sum_{i=0}^{760} c_i x^i$. We shall frequently identify a degree-d polynomial P with a zero-indexed array `P` of length $d+1$, where `P[i]` stores the degree-i coefficient of P.

3.1 Choice of Ring

The Old: The NTRU Prime polynomial modulus $x^{761} - x - 1$ forbids the use of straightforward NTT-based multiplications. Nonetheless, NTT-based multiplications are still applicable if we first lift the computation to $\mathbb{Z}_q[x]$, then map to $\mathbb{Z}_q[x]/\langle Q(x)\rangle$ with $\deg(Q) \geq 1521$ for some new polynomial modulus $Q(x)$ that is more NTT-friendly [Ber01, CHK+21, ACC+21]. FFT-based multiplications limit the choice of $Q(x)$, especially when we keep the integral modulus at $q = 4591$. For instance, to do an incomplete NTT on $\mathbb{Z}_q[x,t]/\langle x^k - t, t^n - 1\rangle \cong \mathbb{Z}_q[x]/\langle x^{nk} - 1\rangle$ requires existence of a n-th primitive root of unity in $\mathbb{Z}_q$, hence $n|(4591 - 1) = 2 \cdot 3^3 \cdot 5 \cdot 17$. Suppose we picked $k = 16$ so as to store each base-case polynomial conveniently in two Neon registers. Since $\deg(Q) = nk \geq 1521$, we have $n \geq 96$. To make n as small as possible, it can only reasonably be $102 = 2 \cdot 3 \cdot 17$. This leads to computation in the ring $\mathbb{Z}_q[x]/\langle x^{1632} - 1\rangle$, as in [HLY24]. Later it was improved to use the smaller ring $\mathbb{Z}_q[x]/\langle \Phi_{17}(x^{96})\rangle$ with truncated Rader's FFT ([Hwa24]). Alternatively, one could use as in [BBCT22] a truncated Schönhage/Nussbaumer NTT to multiply in $\mathbb{Z}_q[x]/\langle (x^{1024} + 1)(x^{512} - 1)\rangle$. Effectively, their algorithm multiplies modulo $x^{1024} + 1$ and modulo $x^{512} - 1$ (both subtasks via Schönhage/Nussbaumer) followed by recovering the product in $\mathbb{Z}_q[x]/\langle (x^{1024} + 1)(x^{512} - 1)\rangle$ via CRT.

The New: We propose a similar approach, except that we use a polynomial modulus of the form $Q(x) = (x^{n_{\text{main}}} - 1)x^{n_{\text{low}}}$ for some $n_{\text{main}} > n_{\text{low}}$. In our case, we use $n_{\text{main}} = 1440$ and $n_{\text{low}} = 81$. We refer to the computation modulo $Q^{\text{main}}(x) = x^{n_{\text{main}}} - 1$ and $Q^{\text{low}}(x) = x^{n_{\text{low}}}$ as the main and the low part respectively, and denote the product in the two parts as $C^{\text{main}}(x) = \sum_{i=0}^{1439} c_i^{\text{main}} x^i$ and $C^{\text{low}}(x) = \sum_{i=0}^{80} c_i^{\text{low}} x^i$.

Using a power of x as a factor has some advantages. CRT is guaranteed to be applicable since $\gcd(x^{n_{\text{main}}} - 1, x^{n_{\text{low}}}) = 1$. In fact, the inverse map will be somewhat simpler than

usual. Because

$$x^{n_{\text{main}}} \equiv 1 \pmod{x^{n_{\text{main}}} - 1} \qquad 1 - x^{n_{\text{main}}} \equiv 0 \pmod{x^{n_{\text{main}}} - 1}$$
$$x^{n_{\text{main}}} \equiv 0 \pmod{x^{n_{\text{low}}}} \qquad 1 - x^{n_{\text{main}}} \equiv 1 \pmod{x^{n_{\text{low}}}},$$

we have $C^{\text{full}} \equiv x^{n_{\text{main}}} C^{\text{main}} + (1 - x^{n_{\text{main}}}) C^{\text{low}} \pmod{Q(x)}$, and

$$c_i^{\text{full}} = \begin{cases} c_i^{\text{low}} & 0 \le i < 81 \\ c_i^{\text{main}} & 81 \le i < 1440 \\ c_{i-1440}^{\text{main}} - c_{i-1440}^{\text{low}} & 1440 \le i < 1521\,, \end{cases} \tag{3}$$

in terms of the coefficients. Moreover, we can control $\deg(Q)$ as needed by adjusting n_{low}. Finally, multiplication in the low part has some special optimization opportunities, since terms with degree at least n_{low} can be safely discarded. In this case, one can can arrive at specialized forms of Karatsuba, Toom-Cook, or inverse NTT in an NTT-based to skip some computations.

3.2 Algorithmic and Computational Overview

In this overview, we outline the end-to-end structure of the whole algorithm. This begins with multiplication in $\mathbb{Z}_q[x]/\langle (x^{1440} - 1)x^{81} \rangle$, which is in turn decomposed into main and low part. To combine the two products into one polynomial of length 1521, we present a way to apply Equation 3 while avoiding excessive data movements. After reducing modulo $x^{761} - x - 1$, the final step is to "freeze" the coefficients into the canonical representation range $[-2295, 2295]$. Each of the stages are implemented with one or more subroutines, most of which have a corresponding C++ function.

3.2.1 Transformation Process of Main Part

For the main part, we use an incomplete NTT with $n = 90, k = 16$. In other words, we start from the isomorphic ring $\mathbb{Z}_q[x,t]/\langle x^{16} - t, t^{90} - 1 \rangle \cong \mathbb{Z}_q[x]/\langle x^{1440} - 1 \rangle$, then use a 90-NTT to map to $\prod_{i=0}^{89} \mathbb{Z}_q[x]/\langle x^{16} - \omega_{90}^i \rangle$. The 90-NTT is in turn decomposed into 10-NTTs and 9-NTTs using Good-Thomas NTT. More concretely, for the 90 inputs $f_0, f_1, \ldots, f_{89}$ to the 90-NTT, by rearranging them into a 2D array of shape 10×9 as Figure 4a, the task is reduced to a 2D NTT. The initial ("Good's") and final ("Ruritanian") permutations are always merged with the computations of another layer.

The 2D NTT can be computed with a layer of column-wise 10-NTTs and a layer of row-wise 9-NTTs. Moreover, by separability of multidimensional DFTs, the two layers can be done in either order. A similar situation exists in reverse, where the 90-iNTT can be implemented as a combination of column-wise 10-iNTTs and row-wise 9-iNTTs, also in either order. Here, we chose to do column-wise 10-NTT and row-wise 9-iNTTs first in the forward and inverse transform respectively. Why so will be explained in Section 3.3.1.

Note to those reading code: in subroutine names, "backward" (abbr. `bwd`) means iNTT (inverse NTT), and "forward" describe the forward transform of the NTT (abbr. `fwd`); Layer 1 and 2 respectively represent a column-wise 10-(i)NTT and a row-wise 9-(i)NTT. So the main part transformations (i.e. excluding the base-case convolutions) consist of 4 subroutines, each named `main_lay{1,2}__{f,b}wd` as appropriate.

3.2.2 Transformation Process of Low Part

Note that a_i, b_i have no effect on the output if $i \ge 81$. Moreover, a_{80}, b_{80} only affect the output with the terms $a_{80} b_0 x^{80}$ and $a_0 b_{80} x^{80}$, which we compute separately and otherwise ignore the degree-80 term of the inputs for now. This leaves us with the

two degree-79 input polynomials $\sum_{i=0}^{79} a_i x^i$ and $\sum_{i=0}^{79} b_i x^i$. Since the degree of their product is now at most 158, we compute it in $\mathbb{Z}_q[x]/\langle x^{160} - 1\rangle$. With a 10-NTT, this is mapped to $\prod_{i=0}^{9} \mathbb{Z}_q[x]/\langle x^{16} - \omega_{10}^i\rangle$. After base-case convolutions and 10-iNTT, we obtain a polynomial of length 160. Discarding the terms with degrees at least 81 gives us $(\sum_{i=0}^{79} a_i x^i)(\sum_{i=0}^{79} b_i x^i) \bmod x^{81}$. Finally adding back the ignored contribution of (a_{80}, b_{80}), we get our desired low part product

$$C^{\text{low}} = AB \bmod x^{81} = \left(\sum_{i=0}^{79} a_i x^i\right)\left(\sum_{i=0}^{79} b_i x^i\right) \bmod x^{81} + (a_{80}b_0 + a_0 b_{80})x^{80}\,. \quad (4)$$

The 10-(i)NTT are implemented as subroutines `low_lay1__{f,b}wd`.

3.2.3 Base-case Weighted Convolutions

At the end of the forward NTT, the main and the low part are decomposed into 90 and 10 base-cases respectively. These are all in the form of size-16 weighted convolutions [CF94], that is, multiplication between two length 16 polynomials, modulo some polynomial of the form $x^{16} - \omega_{90}^i$ for some i. (Note that ω_{10}^i can be rewritten as ω_{90}^{9i}.) The coefficient ω_{90}^i will be referred as the weight of the weighted convolutions. Despite their similarity at first glance, it is better to implement the base-cases of each part as separate subroutines. Indeed, the NTTs located before the base-cases are different in the two parts, and so are the iNTTs located after. Implementing the base-cases separately allows us to plan the modular reductions differently, adaptive to the different input guarantees/output requirements on the coefficient ranges. Additionally, there are different extra operations that we need to perform in this stage. These are some skipped pre/postprocessing steps from the surrounding (i)NTTs, strategically rescheduled and absorbed into computations here. The details and benefits of this rearrangement will be presented in Section 3.3. In summary, the base-case weighted convolutions are implemented as 2 independent subroutines, named `basemul__{main,low}`.

3.2.4 CRT with Minimal Data Movements

As we discussed in Section 3.1, our choice of the two coprime polynomial moduli $x^{1440} - 1$ and x^{81} admits a remarkably simple inverse CRT map. In fact, by rewriting Equation 3 into imperative statements as in Algorithm 1, we see that to combined the products from the two parts, it suffices to (a) store `C_main` into `C_full[81 : 1521]` (in a segmented fashion), (b) store `C_low` into `C_full[0 : 81]` and finally (c) subtract `C_low` from `C_full[1440 : 1521]`. Note that apart from (c), the steps amounts to exactly 1521 element-wise storages.

Algorithm 1 Inverse CRT map

Require: Polynomials $C^{\text{main}}, C^{\text{low}}$ stored as `C_main[1440]`, `C_low[81]`
Ensure: Polynomial C^{full} stored as `C_full[1521]`

```
C_full[81 : 1440] = C_main[81 : 1440]
C_full[1440 : 1521] = C_main[0 : 81]
C_full[0 : 81] = C_low[0 : 81]
C_full[1440 : 1521] -= C_low[0 : 81]
```

Given the simplicity of this process, it is conceivable to merge the steps into the last layers of the inverse transform, i.e. `{main,low}_lay1__bwd`. More specifically, after the computation of 10-iNTT, right when we are about to insert the coefficients into `C_main` and `C_low`, we instead directly store the coefficients into `C_full`. For the remaining step (c), we merge it into `low_lay1__bwd` as well. In the end, compared to implementing the CRT

formula as a separate subroutine, we effectively saved $2 \cdot 1521$ element-wise load and store instructions, or at least $2 \cdot \lceil 1521/32 \rceil = 96$ vectorized instructions `ld1` and `st1` (which can simultaneously operate on 4 vector registers at most), just to give an idea.

We should point out that this is still a slight simplification from what we really implemented in our code: Because of the nature of Neon instructions, where 8 coefficients must be manipulated as a whole, and some further complications like application of Equation 4, there are many special-cases/overlaps/leftovers that need to treated carefully, frequently with dedicated scalar instructions. Nevertheless, this technique still saves considerable amount of data movements, and its correctness is verified in CryptoLine just the same.

3.2.5 Final Reductions and Freezing Coefficients

After the iNTT we need to do three things to compute the final result: multiply by a constant, reduce by modulo $x^{761} - x - 1$, reduce modulo $q = 4591$ to the canonical representative (an operation termed "freezing" as opposed to "squeezing", or lazy reduction, in [BBC+20]). Clearly one should reduce $\sum_{i=0}^{1520} c_i^{\text{full}} x^i$ by $x^{761} - x - 1$ first which gets us $(c_{1520}^{\text{full}} + c_{760}^{\text{full}})x^{760} + (c_{1520}^{\text{full}} + c_{1519}^{\text{full}} + c_{759}^{\text{full}})x^{759} + (c_{1519}^{\text{full}} + c_{1518}^{\text{full}} + c_{758}^{\text{full}})x^{758} + \cdots + (c_{762}^{\text{full}} + c_{761}^{\text{full}} + c_{1}^{\text{full}})x + (c_{761}^{\text{full}} + c_{0}^{\text{full}})$, then do a Barrett multiply by the constant (in our code, $1/170$) followed by a final conditional addition to reduce to between ± 2295.

3.3 Optimization Opportunities

With the general algorithm structure in mind, now we give a deeper dive into the some of the implementation details. In particular, we will focus on the various optimization opportunities noticeable in the computation steps, and describe our ways to exploit these observations to save time.

3.3.1 Zero-skipping

During the forward transform of the main part, the 90 inputs $f_0, f_1, \ldots, f_{89}$ to the 90-NTT come from the zero-padded polynomial of length 1440. Specifically, every 16 consecutive coefficients are clumped together, which form the 90 polynomials of length 16 each. Since the polynomial before the zero-padding was only of length 761, we see that only the first $\lceil 761/16 \rceil = 48$ of them $f_0, f_1, \ldots, f_{47}$ are nonzero. By exploiting the zero elements which constitute almost half of the inputs, it is possible to greatly simplify the forward transform. Now, recall that the 90-NTT was implemented as two layers, and that column-wise 10-NTT and row-wise 9-NTT layer in theory could be done in either order. However, observe that only the preceding layer can make use of the zero elements. Indeed, after its "intermixing" of the coefficients in each column/row, the 2D array will (in general) no longer contain any zeros for the following layer to exploit. Even though the total number of zero elements is the same, depending on the order of the layers, there tends to be a big difference on the saving of computations. After careful comparisons, we concluded that the column-wise 10-NTT makes better use of the zero-inputs, thus should be placed as the frontmost layer. As we will show in Section 3.4.1, zero-inputs in 10-NTTs, especially when they are in specific patterns, will let us skip some modular multiplications.

In `main_lay1__fwd` there are of 9 instances of 10-NTT, each operating on a column of the 10×9 array. As explained in the last paragraph, we know beforehand that 42 elements in the array are actually zeros. From Figure 4b, we see that each 10-NTT instance will contain 4 or 5 zero-inputs depending on the column-index. Specifically, the first 3 columns have 4 zero-inputs each, and the remaining 6 columns have 5 each. However, since each column's zero-inputs are located differently, one would have to implement each 10-NTT instance separately in order to fully utilize all of the zero-inputs. In certain embedded

f_0	f_{10}	f_{20}	f_{30}	f_{40}	f_{50}	f_{60}	f_{70}	f_{80}
f_{81}	f_1	f_{11}	f_{21}	f_{31}	f_{41}	f_{51}	f_{61}	f_{71}
f_{72}	f_{82}	f_2	f_{12}	f_{22}	f_{32}	f_{42}	f_{52}	f_{62}
f_{63}	f_{73}	f_{83}	f_3	f_{13}	f_{23}	f_{33}	f_{43}	f_{53}
f_{54}	f_{64}	f_{74}	f_{84}	f_4	f_{14}	f_{24}	f_{34}	f_{44}
f_{45}	f_{55}	f_{65}	f_{75}	f_{85}	f_5	f_{15}	f_{25}	f_{35}
f_{36}	f_{46}	f_{56}	f_{66}	f_{76}	f_{86}	f_6	f_{16}	f_{26}
f_{27}	f_{37}	f_{47}	f_{57}	f_{67}	f_{77}	f_{87}	f_7	f_{17}
f_{18}	f_{28}	f_{38}	f_{48}	f_{58}	f_{68}	f_{78}	f_{88}	f_8
f_9	f_{19}	f_{29}	f_{39}	f_{49}	f_{59}	f_{69}	f_{79}	f_{89}

(a) All 90 inputs rearranged

f_0	f_{10}	f_{20}	f_{30}	f_{40}				
	f_1	f_{11}	f_{21}	f_{31}	f_{41}			
		f_2	f_{12}	f_{22}	f_{32}	f_{42}		
			f_3	f_{13}	f_{23}	f_{33}	f_{43}	
				f_4	f_{14}	f_{24}	f_{34}	f_{44}
f_{45}					f_5	f_{15}	f_{25}	f_{35}
f_{36}	f_{46}					f_6	f_{16}	f_{26}
f_{27}	f_{37}	f_{47}					f_7	f_{17}
f_{18}	f_{28}	f_{38}						f_8
f_9	f_{19}	f_{29}	f_{39}					

(b) First 48 inputs rearranged

Figure 4: 90-NTT, reduced to a (10×9)-NTT with Good-Thomas. The empty locations in b shows the distribution of zero-elements.

environments, the increased code size is prohibitive. This is a drawback of Good's NTT. For example, in [ACC+21, Section 4.1], the authors used 4 designs of 3-level radix-2 NTT. As the sequence of the required design for each instance are highly irregular, a code generating script was used within their Good's NTT implementation.

To circumvent this, we proposed to cyclically shift the coefficients within each column, then compute the 10-NTT just as usual. Provided we apply a postprocessing step according to the "Time Shifting" property of DFT, we will nevertheless end up with the correct output. More precisely, the property states that if two sequences of length n

$$f[0], f[1], \ldots, f[n-1] \leftrightarrow F[0], F[1], \ldots, F[n-1]$$

form an input/output pair of DFT, then left-shifting f by m results in the pair

$$\begin{aligned} &f[m], f[m+1], \ldots, f[n-1], f[0], \ldots, f[m-1] \\ &\quad \leftrightarrow F[0], \omega_n^{-m} F[1], \omega_n^{-2m} F[2], \ldots, \omega_n^{m} F[n-1]\,. \end{aligned}$$

In NTT parlance, the process of element-wise multiplication of F with $1, \omega_n^{-m}, \omega_n^{-2m}, \ldots, \omega_n^{m}$ is "twisting" the sequence, and the coefficients will be referred as the twisting factors. Hence an intuitive interpretation is that "shifted inputs lead to twisted outputs".

With this technique, we only need two specialized 10-NTT designs. Each of them will handle all the columns with 4 or 5 consecutive zeros, respectively. Moreover, we take this chance to strategically shift the consecutive zeros to appropriate locations, such that the 10-NTT is simplified most effectively. Note that the best choice here depends heavily on the original 10-NTT. For our design, detailed later in Section 3.4.1, it is most beneficial to put the 4 zeros at $f[3], f[4], f[5], f[6]$, and the 5 zeros at $f[3], f[4], f[5], f[6], f[7]$.

We place the postprocessing step at the end of `main_lay1__fwd`. Because of the nonidentical shiftings applied to them, the columns require separate sets of twisting factors each. It is true that this postprocessing step incurs extra modular multiplications. Though we should keep in mind that originally a step of modular reductions would be required around here anyway. As we are replacing it with modular multiplications, the overhead will be comparatively insignificant. Indeed, the merits of simplified NTT and reduced code size more than make up for the postprocessing cost.

In `low_lay1__fwd`, the length 80 input are also padded to length 160, so a similar zero-skipping trick is available. There is only one instance of 10-NTT needed, so we do not have to worry about multiple zero-input patterns for 10-NTT. Nonetheless, we still shift the zeros from $f[5], \ldots, f[9]$ to $f[3], \ldots, f[7]$, which helps to save more modular multiplications, and enables us to reuse the same 10-NTT design in `main_lay1__fwd`. The postprocessing step needed here, i.e. element-wise multiplication of $F[0], \ldots, F[9]$ with the twisting factors $1, \omega_{10}^2, \omega_{10}^4, \ldots, \omega_{10}^{18} = \omega_{10}^8$, is postponed and merged with the base-case convolutions, more specifically the step where we narrow down the result of convolutions from 32-bit to 16-bit.

Compared to `main_lay1__fwd`, as no postprocessing is done within `low_lay1__fwd`, direct outputs of the 10-NTT will have a very wide range of possible values. This makes it impossible to do another round of addition/subtraction without risking overflows in the worst case. It is thus natural to schedule a modular reduction here. However, we argue that in our case, putting this step in the input stage of the base-case subroutine would be the better approach. First, since the weighted convolutions are computed in 32-bit arithmetic, it is possible that the subroutine can handle our large output value without problems. Moreover, even if reduction is really necessary, handling this during its input stage allows the base-case subroutine to choose the best way to perform the reduction, depending on the polynomial modulus for that specific base-case. For example, when 2-NTT-based algorithm is used for the weighted convolution, which is applicable to only half of the base-cases (See Section 3.3.3), some of the coefficients will, before anything else, be 16-bit modular-multiplied with a constant. This entirely eliminates the need to perform reductions on these specific coefficients.

3.3.2 Early-dropping

In `low_lay1__bwd`, the inverse 10-NTT outputs 10 polynomials $f[0], \ldots, f[9]$, each of length 16. Merging all of them will give us a polynomial of length 160. However, since we are computing modulo x^{81}, only the first 6 polynomials $f[0], \ldots, f[5]$ are needed to recover the lowest 81 coefficients. In this case, we can again simplify the inverse NTT, this time by preemptively dropping the intermediate values which are depended only by the to-be-discarded outputs. To make the most out of this, we use the time shifting property once again, though this time we will instead need to preprocess the inputs, i.e. multiply them element-wise with twisting factors. By doing this, we effectively shift the locations of the discarded outputs. As a concrete example, we ultimately want an inverse 10-NTT that discards $f[6], \ldots, f[9]$. We achieve this by using an inverse 10-NTT that discards $f[3], \ldots, f[6]$, plus a preprocessing step on the inputs where they are multiplied element-wise with $1, \omega_{10}^7, \omega_{10}^{14}, \ldots, \omega_{10}^{63} = \omega_{10}^3$. Correspondingly, the preprocessing is also "preponed" and merged with the base-case convolutions.

3.3.3 Base-case Convolutions

For the $90 + 10 = 100$ base-case multiplications, these are all in the form of size-16 weighted convolutions [CF94]. In other words, each of them is a multiplication between two length 16 polynomials, in some ring of the form $\mathbb{Z}_q/\langle x^{16} - \omega_{90}^i \rangle$ for some i. (Note that ω_{10}^i can be rewritten as ω_{90}^{9i}.) As described in Section 2.5, we use 2-NTT or Karatsuba to further reduce the problem into size-8 TMVPs [CCHY23]. More specifically, whenever the weight ω_{90}^i has a square root, i.e. $i = 2k$ is even, we are able to apply 2-NTT and reduce the task to multiplication in $\mathbb{Z}_q/\langle x^8 \pm \omega_{90}^k \rangle$, implemented as 2 TMVPs. For the other cases, we have to fallback to Karatsuba, which is less preferable because 3 TMVPs are required.

3.4 Basic Procedures

With the aforementioned steps to progressively decompose the problem into smaller subtasks, the remaining pieces of the puzzle are the various basic procedures. As the heart of the whole algorithm, these building blocks are critical to the overall performance of the polynomial multiplier.

3.4.1 NTT designs

10-NTT Present in various subroutines, the 10-NTT designs are all based on reduction to (5×2)-NTT via Good-Thomas. Depending the the specific use case, we adaptively choose the first axis to transform on, in similar essence to Section 3.3.1. With the following examples, we aim to illustrate the utility of this flexibility.

Consider the 10-NTTs in `main_lay1__fwd`. By our manipulation in Section 3.3.1, we assume that the inputs $f[3]$ to $f[6]$, sometimes even $f[7]$, are all zeros. According to Good-Thomas, these zeros are rearranged as in Figure 5b and c. In the presence of these zeros, it is better to do 5-NTTs first. The reason is that, while zero-inputs in 2-NTT save additions or subtractions, in 5-NTT they occasionally can further eliminate some of the modular multiplications. For example, as shown in Figure 6, several additions/subtractions and one multiplication are saved if both f_1, f_4 are zeros. Similar saving is also achieved when both f_2, f_3 are zeros. In the end, from the locations of the zeros in each columns, we see that at least a multiplication in the first column for both cases, and a multiplication in the second column for the 5 zeros case are saved. This is the ultimate reason for our placement of the 4 and 5 zeros. As remarked in the last 2 paragraphs of Section 3.3.1, we reuse the 5-zeros design again in `low_lay1__fwd` to profit once more from the eliminated modular multiplications.

f_0	f_5
f_6	f_1
f_2	f_7
f_8	f_3
f_4	f_9

(a) Original input layout

f_0	
	f_1
f_2	f_7
f_8	
	f_9

(b) 4 zeros case

f_0	
	f_1
f_2	
f_8	
	f_9

(c) 5 zeros case

Figure 5: 10-NTT, reduced to a (5×2)-NTT with Good-Thomas.

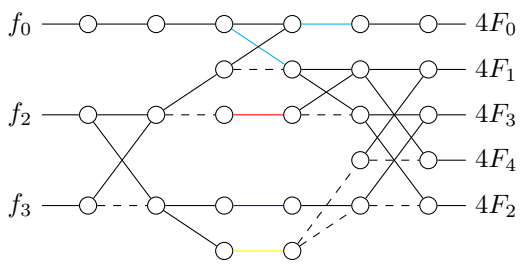


Figure 6: Our radix-5 butterfly, pruned in the case that f_1, f_4 are zeros.

For the 10-iNTTs in `low_lay1__bwd`, on the other hand, we need an inverse 10-NTT that discards $f[3], \ldots, f[6]$. We achieved this by modifying a forward 10-NTT that discards $F[3], \ldots, F[6]$ (See Section 2.4). Recall that in the further reduction to (5×2)-NTT, the

output locations is also rearranged as shown in Figure 7a. (Mind the difference in indexing, cf. Figure 5a.) More importantly, note that this time around, it is the later transformed axis that can benefit from the discardable outputs. This motivates the design where we perform the 5 instances of 2-NTT first, and follow up with 2 differently pruned instances of 5-NTT. From Figure 7b we see that the first and second column needs radix-5 butterfly designs that discards F_2, F_3 and F_0, F_4 respectively. Illustrated in Figure 8, the former butterfly skips a modular multiplication originally in the radix-5 butterfly. This additional saving is the reason behind the manipulation described in Section 3.3.2.

F_0	F_5
F_2	F_7
F_4	F_9
F_6	F_1
F_8	F_3

(a) Original output layout

F_0	
F_2	F_7
	F_9
	F_1
F_8	

(b) 4 discards case

Figure 7: 10-NTT, reduced to a (5×2)-NTT with Good-Thomas.

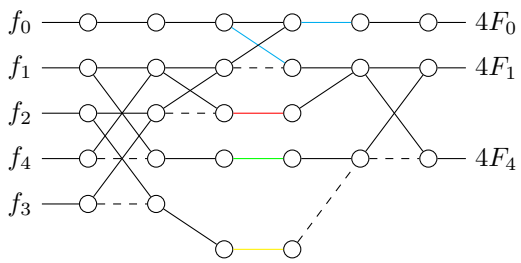
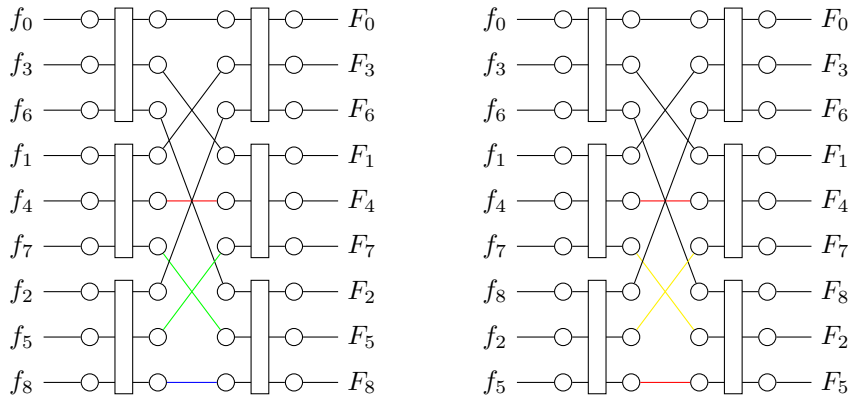


Figure 8: Our radix-5 butterfly, pruned in the case that F_2, F_3 can be discarded.

9-NTT The most typical way to implement a 9-NTT is to use radix-3 Cooley-Tukey with two levels. To reuse the same 3-NTT on the second level, one needs to twist the outputs between the levels. The resulting algorithm is shown in 9a. In our implementation, we use a slightly adjusted variant: By twisting two of the radix-3 "sub-butterflies" as shown in 9b, only 2 distinct constants are required to perform the inter-level twisting, instead of the original 3. This did not really offer a performance boost, but the freed up register slots are nonetheless beneficial.

3.4.2 Variants of Barrett Reduction/Multiplication

One way to summarize the central idea behind Barrett multiplication is as follows. We begin by approximating ab/q using fixed-point arithmetic. Then the rounded value of the fixed-point approximation will be a reasonable approximation for $\lfloor ab/q \rceil$. For example, the standard Barrett multiplication uses $\bar{b} = \lfloor 2^r b/q \rceil$, which is just the conventional fixed-point representation of b/q with r fraction bits. Continuing this, computing $a\bar{b}$ is just the usual way to compute ab/q while retaining r bits of fractional precision, and rounded right-shifting $a\bar{b}$ by r bits exactly corresponds to rounding the fixed-point value to the nearest integer.



(a) Standard radix-3 Cooley-Tukey **(b)** Our variant

Figure 9: 9-NTT designs, where the rectangles represent radix-3 "sub-butterflies", and the red, green, blue and yellow edges represent multiplication by ω_9, ω_9^2, ω_9^4 and ω_9^8 respectively.

In this perspective, it is natural to come up with ways to generalize Barrett's trick: By developing different estimators for the (likely) non-integral value ab/q and/or its nearest integer $\lfloor ab/q \rceil$, different variants of Barrett reduction/multiplication arise, each fitting some specific use-cases. In the following, we depict this idea by presenting some examples employed in our implementation.

Crude Barrett Reduction Cruder approximation will lead to possibly faster reduction but larger output range. When $q = 4591$ (or other number slightly larger than $2^{12} = 4096$), the following is possible. Consider the value $a/4096$. We argue that this is a serviceable approximation for a/q (and for $\lfloor a/q \rceil$ by extension). Quantitatively, this approximation will always overestimate the magnitude by $4591/4096 - 1 \approx 12.1\%$. In view of this, we opted to use $\mathrm{trunc}(a/4096)$ as our estimator, since the truncation or rounding-towards-zero function $\mathrm{trunc}(\cdot)$ helps combat the magnitude overestimation. This gives us Algorithm 2.

With a little effort, the bound $[-4591, 4591]$ on the output is derivable, given that the input fits in a 16-bit integer, i.e. has the bound $[-32768, 32767]$. Though, by building a model in CryptoLine (or simply enumerating every inputs), we obtain a tighter bound $[-4096, 4096]$. While considerably larger than the bound $[-2881, 2881]$ that standard Barrett reduction guarantees (when using 15-bit right-shifts, conveniently computed with `sqrdmulh`), it is perfectly usable in many cases.

Note that this alternative way of reduction uses three instructions to estimate $\lfloor a/q \rceil$, while standard Barrett uses only one `sqrdmulh` instruction. Consequently, this is only faster when the multiplier pipeline is under heavy pressure. Therefore, in different specific use-cases, one should always benchmark and compare the reduction methods carefully, before deciding on the variant to use.

Narrowing Barrett Multiplication At the end of the base-case weighted convolutions, we will have 32-bit intermediate values that need to be reduced to 16-bit ranges, occasionally with a multiply-by-constant step merged together. Since Neon offers a 32-by-32 bit `sqrdmulh` instruction, we can employ Barrett reduction and multiplication for these tasks. With the `uzp1` instruction to extract the low 16-bits of each 32-bit elements, a "narrowing" variant is possible, shown in Algorithm 3 and 4. The key idea is that only the estimation $e = \lfloor a\bar{b}/2^{31} \rceil \approx \lfloor ab/q \rceil$ needs 32-bit arithmetic. After obtaining e, all the remaining computations can be done in 16-bit. This follows from the bound for the original Barrett multiplication $|ab - eq| \leq q$, which guarantees the output to fit in a 16-bit integer.

Algorithm 2 Crude Barrett reduction, with specific modulus 4591

Require:
`v0.8h` $= \vec{x}_0$ where $-2^{15} \leq \vec{x}_{0i} \leq 2^{15} - 1$,
precomputed constant stored as `v1.h[0]` $= 4591$

Ensure: `v0.8h` $\equiv \vec{x}_0 \pmod{4591}$ such that $-4096 \leq$ `v0.h[i]` ≤ 4096

```
sshr v2.8h, v0.8h, #12
cmlt v3.8h, v0.8h, #0
sub v2.8h, v2.8h, v3.8h
mls v0.8h, v2.8h, v1.h[0]
```

Consequently, the computed value will be the same as if 32-bit arithmetic were used. The term "narrowing" is in reference to the naming convention for those Neon instructions where the outputs' bit-widths are half of the inputs' [ARM13, Section 3.2.2][Zha15].

Algorithm 3 Narrowing Barrett reduction, with general modulus q

Require:
`v0.4s ++ v1.4s` $= \vec{x}_0$ where $-2^{31} \leq \vec{x}_{0i} \leq 2^{31} - 1$,
precomputed constants stored as `v2.s[0]` $= \lfloor 2^{31}/q \rceil$, `v2.h[2]` $= q$

Ensure: `v3.8h` $\equiv \vec{x}_0 \pmod{q}$ such that $-q \leq$ `v3.h[i]` $\leq q$

```
uzp1 v3.8h, v0.8h, v1.8h
sqrdmulh v0.4s, v0.4s, v2.s[0]
sqrdmulh v1.4s, v1.4s, v2.s[0]
uzp1 v4.8h, v0.8h, v1.8h
mls v3.8h, v4.8h, v2.h[2]
```

Algorithm 4 Narrowing Barrett multiplication, with general modulus q

Require:
`v0.4s ++ v1.4s` $= \vec{x}_0$ where $-2^{31} \leq \vec{x}_{0i} \leq 2^{31} - 1$,
precomputed constants stored as `v2.s[0]` $= \lfloor 2^{31}b/q \rceil$, `v2.h[2]` $= q$, `v2.h[3]` $= b$

Ensure: `v3.8h` $\equiv b\vec{x}_0 \pmod{q}$ such that $-q \leq$ `v3.h[i]` $\leq q$

```
uzp1 v3.8h, v0.8h, v1.8h
sqrdmulh v0.4s, v0.4s, v2.s[0]
sqrdmulh v1.4s, v1.4s, v2.s[0]
uzp1 v4.8h, v0.8h, v1.8h
mul v3.8h, v3.8h, v2.h[3]
mls v3.8h, v4.8h, v2.h[2]
```

4 Verification

C++ intrinsic functions are used to employ Neon instructions in our implementation (Section 3). Compiler optimization moreover is enabled to reschedule our implementation. With algorithmic optimizations described in the previous section, the correctness of our implementation is not obvious and demands proofs. In order to ensure the correctness of our implementation, we use the CryptoLine tool to formally verify the functional correctness of our implementation.

Concretely, we verify that our compiler-optimized implementation correctly computes the product AB in $\mathbb{Z}_q[x]/\langle x^{761} - x - 1\rangle$ with $q = 4591$ for all input polynomials $A =$

$\sum_{i=0}^{760} a_i x^i$ and $B = \sum_{i=0}^{760} b_i x^i$ with $a_i, b_i \in [-2295, 2295]$ for $0 \leq i < 761$. Our end-to-end verification ensures all ideas in Section 3 are implemented correctly but also correct themselves.

4.1 Variant of Barrett Multiplication

Our Barrett multiplication approximates $\lfloor x/4591 \rceil$ by $\mathrm{trunc}(x/4096)$ (Section 3.4.2). Its output range thus may differ from standard Barrett multiplication. Moreover, our optimized implementation minimizes the number of reductions. Our Neon implementation meanwhile stores a coefficient in $\mathbb{Z}_q$ in a 16-bit word. It is crucial to ensure the absence of unexpected overflow during computation.

In our verification, we use signed 16-bit variables to store all intermediate computation results. The CryptoLine tool automatically checks the absence of unexpected overflow for all inputs. The verification tool in turn invokes the Satisfiability Modulo Theories (SMT) over Quantified-Free Bit-Vector Theory (QFBV) solver Boolector to verify all computation is safe without unexpected overflow.

Our variant of Barrett multiplication is implemented by three Neon instructions (`sqrdmulh`, `shl`, and `mls`). Although no unexpected overflow occurs, it does not imply the modular multiplication is computed correctly. The CryptoLine tool moreover uses the linear integer arithmetic solver isl to verify the correctness of all modular multiplication.

4.2 NTTs in Main and Low Parts

After ensuring the correctness of modular multiplication, our next task is to verify the 90-NTT detailed in Section 3.2.1. More specifically, we would like to verify our implementation of Good-Thomas algorithm for the input polynomial of degree 1439.

Let the input polynomial $\bar{A} = \sum_{i=0}^{1439} \bar{a}_i x^i \in \mathbb{Z}_q[x]/\langle x^{1440} - 1\rangle$. Since $1440 = 10 \cdot 9 \cdot 16$,

$$\mathbb{Z}_q[x]/\langle x^{1440} - 1\rangle \cong (\mathbb{Z}_q[x, z]/\langle x^{16} - yz, z^9 - 1\rangle)[y]/\langle y^{10} - 1\rangle.$$

After 10-NTT with respect to y, we obtain 10 polynomials $\bar{\Gamma}_k$ such that

$$\bar{\Gamma}_k = \sum_{j=0}^{8} \bar{\gamma}_{9k+j} z^j \text{ for } 0 \leq k < 10$$

where $\bar{\gamma}_j \in \mathbb{Z}_q[x]$ is of degree 15 for $0 \leq j < 90$. And CryptoLine verifies

$$4\bar{A} \equiv \bar{\Gamma}_k \bmod [q, x^{16} - yz, y - \omega_{10}^k, z^9 - 1] \text{ for } 0 \leq k < 10.$$

Similarly, we obtain 90 degree-15 polynomials $\bar{\alpha}_j \in \mathbb{Z}_q[x]$ for $0 \leq j < 90$ after 9-NTT with respect to z. Our verification on the main part concludes by proving

$$8\bar{A} \equiv \bar{\alpha}_j \bmod [q, x^{16} - yz, y - \omega_{10}^{\lfloor j/9 \rfloor}, z - \omega_9^{j \bmod 9}] \text{ for } 0 \leq j < 90.$$

The inverse 90-NTT is verified similarly. Consequently, our implementation correctly transforms the degree-1439 input polynomial into 90 degree-15 output polynomials through Good-Thomas algorithm.

The 10-NTT in the low part is similar (Section 3.2.2). Let $\underline{A} = \sum_{i=0}^{79} \underline{a}_i x^i$ be the degree-79 input polynomial. Observe $\mathbb{Z}_q[x]/\langle x^{160} - 1\rangle \cong \mathbb{Z}_q[x]/\langle x^{16} - y, y^{10} - 1\rangle$. We obtain 10 degree-15 polynomials $\underline{\alpha}_j \in \mathbb{Z}_q[x]$ for $0 \leq j < 10$ after 10-NTT of the low part. Moreover, one could verify the following condition with CryptoLine:

$$4\underline{A} \equiv \omega_{10}^{2j} \underline{\alpha}_j \bmod [q, x^{16} - y, y - \omega_{10}^j] \text{ for } 0 \leq i < 10.$$

Note that we had to include the twisting factors ω_{10}^{2j} into the equation. This is a direct consequence of skipping the postprocessing in `low_lay1__fwd`, discussed in the last two paragraphs of Section 3.3.1.

We nevertheless did not verify the condition explicitly. Our end-to-end verification shows that CryptoLine is able to infer the condition by itself. Less human guidance is needed in practice. To summarize, our verification proves that all optimizations in 90-NTT such as Good-Thomas algorithm, zero skipping, and early dropping are all implemented correctly.

4.3 Polynomial Multiplication

Let $A, B \in \mathbb{Z}_q[x]/\langle x^{761} - x - 1\rangle$ be the input polynomials. We obtain their main parts in $\mathbb{Z}_q[x]/\langle x^{1440} - 1\rangle$ through 90-NTTs. Precisely,

$$\begin{array}{rcl} 8A & \equiv & \overline{\alpha}_j \bmod [q, x^{16} - yz, y - \omega_{10}^{\lfloor j/9 \rfloor}, z - \omega_9^{j \bmod 9}] \text{ for } 0 \le j < 90 \\ 8B & \equiv & \overline{\beta}_j \bmod [q, x^{16} - yz, y - \omega_{10}^{\lfloor j/9 \rfloor}, z - \omega_9^{j \bmod 9}] \text{ for } 0 \le j < 90 \end{array}$$

After base-case convolution (Section 3.2.3), we obtain $\overline{\gamma}_j \in \mathbb{Z}_q[x]/\langle x^{16} - 1\rangle$ such that

$$64AB = \overline{\gamma}_j \bmod [q, x^{16} - yz, y - \omega_{10}^{\lfloor j/9 \rfloor}, z - \omega_9^{j \bmod 9}] \text{ for } 0 \le j < 90.$$

After the 90-iNTT, we have $\overline{C} \in \mathbb{Z}_q[x]/\langle x^{16} - yz, y^{10} - 1, z^9 - 1\rangle$ such that

$$\overline{C} \equiv 170AB \bmod [q, x^{16} - yz, y^{10} - 1, z^9 - 1].$$

Now let $\underline{A} = A \bmod x^{80}$ and $\underline{B} = B \bmod x^{80}$ be the low parts of A and B respectively. We obtain $\underline{C} \in \mathbb{Z}_q[x]/\langle x^{80}\rangle$ such that

$$\underline{C} \equiv \underline{AB} \bmod x^{80}.$$

From $\overline{C}$ and $\underline{C}$, we compute C^{main} and C^{low} respectively (Section 3.1). Our implementation obtains the polynomial C^{full} of 1520 coefficients such that

$$\begin{array}{rcl} C^{\text{full}} & \equiv & 170AB \bmod [q, x^{81}] \\ C^{\text{full}} & \equiv & 170AB \bmod [q, x^{1440} - 1]. \end{array}$$

Finally, the product $C = \sum_{i=0}^{760} c_i x^i \in \mathbb{Z}_q[x]/\langle x^{761} - x - 1\rangle$ is obtained from C^{full} (Section 3.2.5) that

$$\begin{array}{rcl} C & \equiv & 170AB \bmod [q, x^{761} - x - 1] \\ c_i & \in & [-2295, 2295] \qquad \text{for } 0 \le i < 761 \end{array} \tag{5}$$

CryptoLine verifies all conditions in (5).

4.4 Compiler Optimization

The verification results in the last sub-section do not include full compiler optimizations. CryptoLine is designed to verify assembly programs. We use `gcc` to generate assembly codes from our C++ implementation and verify the generated assembly codes. However, assembly instructions are rescheduled drastically after full compiler optimizations. We were unable to verify assembly codes generated with the compiler flag `-O3`. Instead, our verification is only valid for the assembly codes generated with the flag `-O1`.

In order to verify assembly codes generated by full compiler optimizations, we apply the technique in [LLS+24]. After the suboptimal assembly codes are verified with CryptoLine.

We use the logic equivalence checking in CryptoLine to verify the suboptimal and the optimal assembly codes are functionally equivalent. That is, both codes will always compute the same output polynomial given the same input polynomials. Since the output polynomial of the assembly codes generated with `-O1` flag is correct, the output polynomial of the assembly codes generated with `-O3` flag is also correct. Our optimized implementation of polynomial multiplication for NTRU Prime is formally verified.

5 Results

We evaluate our implementation and report our verification of the optimized polynomial multiplication for NTRU Prime in this section.

5.1 Performance of Polynomial Multiplication

The performance of our NTRU Prime polynomial multiplication is evaluated on Raspberry Pi 3, Pi 4, and Pi 5 with ARM Cortex-A53, Cortex-A72, and Cortex-A76 respectively. We use `gcc 12.2.0` to compile our C++ implementation with the optimization flag `-O3`. Table 1 compares our new algorithm with the state of the art in [Hwa24] by CPU cycles.

Table 1: Cycle count for multiplication in $\mathbb{Z}_q[x]/\langle x^{761} - x - 1\rangle$

Benchmark Environment	[Hwa24]	Ours	Speedup
Cortex-A53	50689	39018	1.30x
Cortex-A72	31987	26772	1.19x
Cortex-A76	26295	21165	1.24x

Our evaluation shows that our optimized polynomial multiplication is faster than [Hwa24] by at least 19%. On Raspberry Pi 3, the speedup of 30% is attained. Our NTT-based multiplications over a polynomial ring with an atypical factor X^m can be significantly faster than the best prior algorithm for `sntrup761`.

Table 2: Cycle counts per subroutine, on Cortex-A72

Subroutine	#Calls	Cycles	Subroutine	#Calls	Cycles
`mainmul`	1	23051	`lowmul`	1	1741
`main_lay1__fwd`	2	2610	`low_lay1__fwd`	2	164
`main_lay2__fwd`	2	2345			
`basemul__main`	1	8142	`basemul__low`	1	1173
`main_lay2__bwd`	1	2358	`low_lay1__bwd`	1	214
`main_lay1__bwd`	1	2619			
`mod_poly`	1	989	`freeze`	1	967

To identify the bottleneck of the polynomial multiplication, Table 2 gives the cycle counts for each subroutine on Raspberry Pi 4. The column *#Calls* indicates how many times the subroutine is invoked; the column *Cycles* shows the cycle counts per invocation. As expected, the main part takes about 86% of the total cycles (26772). About 30% of the time is used on the base-case convolutions in the main part.

5.2 Cost of Verification

Table 3 details the verification time for the polynomial multiplication in seconds. The column *Safety* shows the time for overflow checking. The columns *RA* (range assertion) and

Table 3: Verification time for the correctness of `-O1` models, in seconds

Function	Safety	RA	RS	AA	AS
`basemul__low`	48.27	0.03	14.20	56.36	49.78
`basemul__main`	584.40	4.30	491.09	1290.12	1923.39
`low_lay1__bwd`	632.69	1028.02	283.11	3.12	1.92
`low_lay1__fwd`	91.67	177.31	4.50	2.47	2.60
`main_lay1__bwd`	271.28	100.63	122.85	3460.13	267.00
`main_lay1__fwd`	1086.61	2.73	1236.63	684.17	191.31
`main_lay2__bwd`	1902.30	0.49	156.55	777.60	177.99
`main_lay2__fwd`	1800.14	0.62	187.47	4491.65	223.34

RS (range specification) give the time for range analysis. And *AA* (algebraic assertion) and *AS* (algebraic specification) show the time for algebraic property verification. Verification of the main part occupies a large portion of time. Notably, the algebraic properties in 10-NTT (`main_lay2__fwd_inplace`) and 9-iNTT (`main_lay1__bwd_insert`) take more than 50 minutes to verify. One could improve the verification time by adding more proof hints manually [HLS+22]. We leave it as a future work.

Our equivalence checking calls the `abc` toolkit through CryptoLine. Note that between the optimization flags `-O1` and `-O3`, the `gcc` compiler does a lot of complex transformations and rescheduling for code generation, which explains the week-long wall time there. However, equivalence checking is mostly automatic and requires little human guidance. Manually adding more hints can reduce the verification time and is left as a future work.

Table 4: Verification time for the equivalence between `-O3` and `-O1` models, in seconds

Function	Equivalence	Function	Equivalence
`basemul__low`	586994.63	`main_lay1__bwd`	911.66
`basemul__main`	3642.91	`main_lay1__fwd`	829.96
`low_lay1__bwd`	24.81	`main_lay2__bwd`	835.79
`low_lay1__fwd`	26.52	`main_lay2__fwd`	827.69

6 Conclusions and Future Work

We have described how one obtain a high-performance, high-assurance polynomial multiplication useful for NTRU Prime. We hope that the detailed description also serves as a documentation for our code and an instruction instrument as well as an engineering manual for people interested in unorthodox NTT techniques.

For future work, we can still consider various possible optimizations; one possibility is adapting the radix-3 butterfly used in [HY22, HLY24]. We can also adapt the optimizations to other NTRU variant cryptosystems.

Acknowledgments

The authors would lke to thank their institution for the Grand Challenge Project AS-GCP-114-M01 and the Taiwanese National Science and Technology Council for grants NSTC 111-2221-E-001-014-MY3, 113-2221-E-001-023-MY3, and 114-2221-E-001-022-MY3.

References

[ABB+23] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, Antoine Séré, and Pierre-Yves Strub. Formally verifying Kyber episode IV: Implementation correctness. *IACR TCHES*, 2023(3):164–193, 2023.

[ACC+21] Erdem Alkim, Dean Yun-Li Cheng, Chi-Ming Marvin Chung, Hülya Evkan, Leo Wei-Lun Huang, Vincent Hwang, Ching-Lin Trista Li, Ruben Niederhagen, Cheng-Jhih Shih, Julian Wälde, and Bo-Yin Yang. Polynomial multiplication in NTRU prime. *IACR TCHES*, 2021(1):217–238, 2021.

[AOB+24] José Bacelar Almeida, Santiago Arranz Olmos, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Cameron Low, Tiago Oliveira, Hugo Pacheco, Miguel Quaresma, Peter Schwabe, and Pierre-Yves Strub. Formally verifying kyber - episode V: Machine-checked IND-CCA security and correctness of ML-KEM in EasyCrypt. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 384–421. Springer, Cham, August 2024.

[ARM13] ARM. *NEON Programmer's Guide*, 2013. https://developer.arm.com/documentation/den0018/a.

[BBC+20] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, Chitchanok Chuengsatiansup, Tanja Lange, Adrian Marotzke, Bo-Yuan Peng, Nicola Tuveri, Christine van Vredendaal, and Bo-Yin Yang. NTRU Prime. Technical report, National Institute of Standards and Technology, 2020. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions.

[BBCT22] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. OpenSSLNTRU: Faster post-quantum TLS key exchange. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 845–862. USENIX Association, August 2022.

[BBF+21] Manuel Barbosa, Gilles Barthe, Xiong Fan, Benjamin Grégoire, Shih-Han Hung, Jonathan Katz, Pierre-Yves Strub, Xiaodi Wu, and Li Zhou. EasyPQC: Verifying post-quantum cryptography. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 2564–2586. ACM Press, November 2021.

[Ber01] Daniel J. Bernstein. Multidigit multiplication for mathematicians. 2001. https://cr.yp.to/papers.html#m3.

[BHK+22] Hanno Becker, Vincent Hwang, Matthias J. Kannwischer, Bo-Yin Yang, and Shang-Yi Yang. Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1. *IACR TCHES*, 2022(1):221–244, 2022.

[CCHY23] Han-Ting Chen, Yi-Hua Chung, Vincent Hwang, and Bo-Yin Yang. Algorithmic views of vectorized polynomial multipliers - NTRU. In Anupam Chattopadhyay, Shivam Bhasin, Stjepan Picek, and Chester Rebeiro, editors, *INDOCRYPT 2023, Part II*, volume 14460 of *LNCS*, pages 177–196. Springer, Cham, December 2023.

[CF94] Richard Crandall and Barry Fagin. Discrete weighted transforms and large-integer arithmetic. *Math. Comput.*, 62(205):305–324, January 1994.

[CHK+21] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jhih Shih, and Bo-Yin Yang. NTT multiplication for NTT-unfriendly rings. *IACR TCHES*, 2021(2):159–188, 2021.

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.

[Goo58] I. J. Good. The interaction algorithm and practical fourier analysis. *J. of the Royal Statistical Society, Series B*, 20(2):361–372, 1958.

[HKS23] Vincent Hwang, YoungBeom Kim, and Seog Chung Seo. Barrett multiplication for dilithium on embedded devices. Cryptology ePrint Archive, Report 2023/1955, 2023.

[HLS+22] Vincent Hwang, Jiaxiang Liu, Gregor Seiler, Xiaomu Shi, Ming-Hsien Tsai, Bow-Yaw Wang, and Bo-Yin Yang. Verified NTT multiplications for NISTPQC KEM lattice finalists: Kyber, SABER, and NTRU. *IACR TCHES*, 2022(4):718–750, 2022.

[HLY24] Vincent Hwang, Chi-Ting Liu, and Bo-Yin Yang. Algorithmic views of vectorized polynomial multipliers - NTRU prime. In Christina Pöpper and Lejla Batina, editors, *ACNS 2024, Part II*, volume 14584 of *LNCS*, pages 24–46. Springer, Cham, March 2024.

[Hwa24] Vincent Hwang. Pushing the limit of vectorized polynomial multiplications for NTRU prime. In Yannan Li Tianqing Zhu, editor, *ACISP 24, Part II*, volume 14896 of *LNCS*, pages 84–102. Springer, Singapore, July 2024.

[HY22] Chenar Abdulla Hassan and Oğuz Yayla. Radix-3 NTT-based polynomial multiplication for lattice-based cryptography. Cryptology ePrint Archive, Report 2022/726, 2022.

[HZZ+22] Junhao Huang, Jipeng Zhang, Haosong Zhao, Zhe Liu, Ray C. C. Cheung, Çetin Kaya Koç, and Donglong Chen. Improved plantard arithmetic for lattice-based cryptography. *IACR TCHES*, 2022(4):614–636, 2022.

[LLS+24] Li-Chang Lai, Jiaxiang Liu, Xiaomu Shi, Ming-Hsien Tsai, Bow-Yaw Wang, and Bo-Yin Yang. Automatic verification of cryptographic block function implementations with logical equivalence checking. In Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas, editors, *ESORICS 2024, Part IV*, volume 14985 of *LNCS*, pages 377–395. Springer, Cham, September 2024.

[LS19] Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly fast NTRU using NTT. *IACR TCHES*, 2019(3):180–201, 2019.

[Rad68] C. M. Rader. Discrete fourier transforms when the number of data samples is prime. *Proceedings of the IEEE*, 56(6):1107–1108, 1968.

[Sei18] Gregor Seiler. Faster AVX2 optimized NTT multiplication for ring-LWE lattice cryptography. Cryptology ePrint Archive, Report 2018/039, 2018.

[Tho63] L. H Thomas. *Applications of Digital Computers*, chapter Using a computer to solve problems in physics. Ginn, Boston, 1963.

[Zha15] Yang Zhang. ARM NEON programming quick reference, 2015. https://community.arm.com/arm-community-blogs/b/operating-systems-blog/posts/arm-neon-programming-quick-reference.